

Which Fix Should We Test? Budgeted Causal Verification of LLM Failure Attributions for Self-Improving Agents

Anonymous ACL submission

Abstract

Self-improving LLM agents increasingly rely on an LLM to read failed trajectories, name a root cause, and decide which part of the agent to change. These attributions are cheap but unreliable, and checking them by replaying the agent with a candidate fix is reliable but expensive. We ask which attributions are worth checking under a fixed replay budget. We cast the problem as learning a causal bipartite graph from failure categories to modifiable agent components, where each edge is an unknown intervention effect. Our method, CARVE, places a Bayesian model on this graph that separates whether a component is a cause from how good a particular patch is, uses LLM attributions as a prior rather than a verdict, and accounts for noisy category labels. It chooses the next intervention (which category, component and patch, and whether to run a cheap single-step check or a full replay) by maximising expected decision value per unit cost. Two propositions characterise when the LLM prior saves replays and when cheap checks pay off. In a controlled testbed with known ground truth, CARVE recovers 79% of the oracle’s repairable failure mass at a budget of 50 replays, against 66% for uncertainty sampling with the same cheap checks and at most 52% for do-then-verify baselines, and in a multi-round self-improvement loop it reaches in two rounds the success rate that the strongest baseline reaches in four. Most of its advantage comes from how it uses cheap checks, so it fails when check errors correlate with patch quality unless that correlation is modelled. Real-agent experiments are pending; we release the method and a fault-injected tool-use environment for them.

1 Introduction

A growing family of systems improves LLM agents from their own failures: the agent runs, an analyst model reads the failed trajectories, and its diagnosis drives a change to a prompt, a tool, or the

orchestration code (Shinn et al., 2023; Zhao et al., 2024; Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026). The diagnosis step has become a research topic in its own right under the name *failure attribution* (Zhang et al., 2025b, 2026; Zhu et al., 2025). Two findings from that literature motivate this paper.

First, LLM attribution is unreliable. On the recent Who&When Pro benchmark the strongest model reaches 74% step-level accuracy on text traces but only 22% F1 on identifying the error mode, and accuracy falls to roughly 50% on traces longer than 12k tokens (Liu et al., 2026). Models tend to label failures by their visible symptom rather than their cause. A self-improvement loop that trusts these labels will often change the wrong component, and a wrong change can hurt: accurate failure prediction does not imply that acting on it helps (Vasudev et al., 2026).

Second, attributions can be checked by intervention. If a diagnosis says that a tool description caused a failure, one can patch that description, replay the trajectory from the failing step, and see whether the task now succeeds (Ma et al., 2025; Lin et al., 2026; Bonagiri et al., 2026; Shah, 2026). This is the agent analogue of a *do*(·) experiment (Pearl, 2009), and it is far more reliable than reading the trace. It is also expensive: each check costs a partial agent rollout, outcomes are stochastic so one replay is rarely conclusive, and a failed check is ambiguous because the patch itself may simply be bad.

Existing intervention-based methods verify one hypothesis at a time, usually with a fixed number of replays per hypothesis. In a self-improvement loop the question is different. There are many failure categories and many components that could be changed, the replay budget is fixed, and what matters is not the accuracy of any single diagnosis but how much of the agent’s failure mass is repaired at the end. We therefore ask: *given a budget of*

interventions, which attributions should be verified, with which patch, and how cheaply?

We make the following contributions.

- We formulate budgeted verification as learning a *category–component causal graph* whose edges are unknown intervention effects, and we separate the causal question (is component j a cause of category i ?) from the engineering question (is this particular patch any good?) by giving each cell several candidate patches (§3).
- We propose CARVE (Cost-Aware Replay for Verifying Edges), a Bayesian method that uses LLM attributions as a prior whose strength is set by the analyst’s audited accuracy, models label noise and spurious recovery, and selects interventions across cells, patches and two fidelities (a cheap single-step check and a full replay) with a multi-step knowledge-gradient rule divided by cost (§4).
- We give two propositions that say when the LLM prior saves replays and when a cheap check is worth running, in terms of quantities a practitioner can estimate (§5).
- In a controlled testbed with known ground truth, CARVE needs about half the budget of the strongest baseline to reach the same repaired failure mass, is never worse than a flat prior even when LLM attributions are nearly useless, and converts that efficiency into faster agent improvement inside a multi-round loop. Stress tests identify its main failure mode: single-step checks whose errors correlate with patch quality (§6). We also describe ShopDesk, a deterministic tool-use environment with injected faults that provides ground truth for real-LLM studies, together with the full pipeline (§7); real-agent results are not yet included in this draft.

2 Related Work

Failure attribution for LLM agents. Zhang et al. (2025b) introduced the Who&When benchmark and compared all-at-once, step-by-step and binary-search prompting for locating the responsible agent and step; Liu et al. (2026) extended it to longer and multimodal traces and found that attribution remains far from solved. AgenTracer uses counterfactual replay and fault injection to build training data for a small attribution model (Zhang

et al., 2026); AgentDebug pairs an error taxonomy with targeted corrective feedback (Zhu et al., 2025); ErrorProbe traces dependencies backwards and keeps a verified memory of error patterns (Li et al., 2026). Taxonomies such as MAST (Cemri et al., 2025) give attribution a fixed vocabulary, and AdaMAST induces a system-specific taxonomy from traces whose axes map to intervention points (Cemri et al., 2026). AdaMAST validates its taxonomy by agreement between LLM annotators; we instead treat the mapping from categories to components as a causal claim to be checked by intervention.

Verification by intervention. DoVer generates hypotheses from logs and validates each by editing a message or plan and re-running, with several runs per intervention (Ma et al., 2025). RE-FLECT tests diagnosis-specific patches by targeted replay and feeds verified outcome changes back into attribution (Lin et al., 2026). CausalFlow scores steps by their causal responsibility under counterfactual interventions (Bonagiri et al., 2026), and Causal Agent Replay formalises step interventions in a structural causal model with a point-of-commitment rule (Shah, 2026). AgentDebugX provides a toolkit with policy-gated reruns and a store of failure–diagnosis–repair bundles (Zhu et al., 2026). All of these operate per trajectory or per hypothesis and do not decide how to allocate a shared budget across hypotheses, patches and levels of fidelity, which is our focus. Our method can use any of them as the replay primitive.

Self-improving agents. Reflexion, Self-Refine and ExpeL turn feedback into verbal lessons (Shinn et al., 2023; Madaan et al., 2023; Zhao et al., 2024); prompt optimisers such as ProTeGi, OPRO, TextGrad and DSPy use failures or textual feedback to rewrite prompts (Pryzant et al., 2023; Yang et al., 2024; Yuksekgonul et al., 2025; Khattab et al., 2024); ADAS, the Darwin Gödel Machine and GEPA search over agent designs or prompts guided by evaluation and failure logs (Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026). HarnessFix attributes failures to harness artefacts and repairs them (Chen et al., 2026). Bayesian-Agent keeps Beta posteriors over the reliability of skills and applies threshold rules to patch, split or retire them (Wu et al., 2026); its posterior is over skills rather than causal edges, and actions are chosen by fixed thresholds rather than by the value of information. CARVE is complementary: it de-

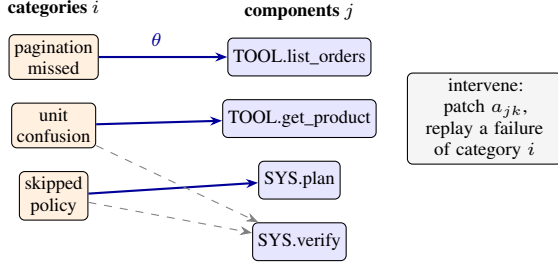


Figure 1: The category–component graph. Solid edges are true causal edges; dashed ones are plausible but wrong attributions (e.g. blaming verification for a unit error). Each intervention patches one component and replays one failure.

cides which proposed fixes to trust before they are applied.

Experimental design. Our selection rule is the KG(*) variant of the knowledge gradient (Frazier et al., 2008; Frazier and Powell, 2010), within Bayesian experimental design (Chaloner and Verdinelli, 1995). Related problems include multi-fidelity bandits and optimisation (Kandasamy et al., 2016; Poloczek et al., 2017), thresholding and best-arm identification (Locatelli et al., 2016; Audibert et al., 2010; Russo, 2016), sequential testing (Wald, 1945), active causal structure learning (Tong and Koller, 2001; Hauser and Bühlmann, 2014) and causal bandits (Lattimore et al., 2016). In software engineering, delta debugging isolates failure-inducing changes by experiment (Zeller and Hildebrandt, 2002), and program-repair work warns that a patch passing a failing test may not fix the fault (Smith et al., 2015), which our separation of edges from patch quality reflects. Our contribution is not a new design criterion but a formulation of agent self-improvement in which these tools apply, with a model of its specific nuisances (bad patches, noisy labels, spurious recovery, LLM priors).

3 Problem Formulation

An agent is built from *components* $j \in \{1, \dots, J\}$: sections of its system prompt, tool descriptions, tool implementations, orchestration settings. Its failures have been grouped into *categories* $i \in \{1, \dots, I\}$ by an LLM, with f_i the share of failures in category i . For every cell (i, j) an analyst LLM can propose K candidate patches a_{ijk} that rewrite component j to address category i .

Interventions. An intervention (i, j, k, ϕ) samples a failed trajectory labelled i , applies patch

a_{ijk} , and observes a binary outcome at fidelity ϕ . A *full replay* ($\phi = F$) resumes the episode from the attributed failing step and records whether the task now succeeds. A *single-step check* ($\phi = S$) regenerates only the failing step and asks a judge whether the diagnosed error is gone. They cost c_F and $c_S \ll c_F$.

Edges and patch quality. We write $e_{ij} \in \{0, 1\}$ for whether component j is a cause of category i , meaning some change to j repairs a non-trivial fraction of its failures, and $q_{ijk} \in [0, 1]$ for the repair probability of patch k when $e_{ij} = 1$. This separation matters because a negative replay outcome is ambiguous: either j is the wrong component or a_{ijk} is a bad patch. Testing several patches per cell resolves the ambiguity.

Objective. After spending at most budget B , the method accepts a set of cells $\mathcal{A}$ and, for each, a patch k_{ij}^* . Accepting a true edge repairs a fraction q_{ijk}^* of category i ’s failures; accepting a false edge costs a regression c_{fp} , reflecting that unnecessary changes perturb behaviour on tasks that used to succeed (Vasudev et al., 2026). We measure

$$G(\mathcal{A}) = \sum_i f_i \left(1 - \prod_{j \in \mathcal{A}_i^+} (1 - q_{ijk_{ij}^*}) \right) - c_{fp} |\{(i, j) \in \mathcal{A} : e_{ij} = 0\}|, \quad (1)$$

where $\mathcal{A}_i^+ = \{j : (i, j) \in \mathcal{A}, e_{ij} = 1\}$. This is the net failure mass repaired; we report it as a fraction of the oracle value obtained by accepting exactly the true edges with their best patches.

4 CARVE

4.1 Observation model

For a trajectory labelled i and patch a_{ijk} , the probability that a full replay succeeds is

$$p_{ijk} = \begin{cases} \lambda q_{ijk} + (1 - \lambda)b & e_{ij} = 1, \\ b & e_{ij} = 0, \end{cases} \quad (2)$$

where b is the spurious recovery rate (stochastic re-generation sometimes succeeds regardless of the patch) and λ is the accuracy of the category labels used to sample trajectories; a mislabelled trajectory is treated as unaffected by the patch. A single-step check returns $z = 1$ with probability $sp + \epsilon(1 - p)$, where s and ϵ are the sensitivity and false-positive rate of the check relative to the full outcome. All four nuisance parameters can be estimated from a small calibration set on which both fidelities and a human audit are run.

4.2 Attribution-informed prior

Let n_{ij} be the number of failures in category i that the analyst attributed to component j , and $\hat{r}$ the analyst’s attribution accuracy estimated on a small audited sample. With d the expected number of causal components per category and $\rho = d/J$ the base edge density, we set

$$\pi_{ij} = (1 - \hat{r})\rho + \hat{r} \min(1, d n_{ij}/n_i), \quad (3)$$

so that an unreliable analyst ($\hat{r} \rightarrow 0$) yields the flat prior ρ and a reliable one concentrates mass on the components it names. Eq. 3 is a heuristic interpolation, not a calibrated model; with a larger audit, π could instead be fitted by logistic regression on audited attributions. Patch quality has prior $q_{ijk} \sim \text{Beta}(\alpha_q, \beta_q)$.

4.3 Posterior

Cells are independent given the data. For each cell we keep the posterior log-odds of e_{ij} and, for each patch, a posterior over q_{ijk} on a fixed grid of $G = 101$ points. With $\mathcal{D}_{ijk}$ the outcomes observed for patch k ,

$$\begin{aligned} \text{logit } P(e_{ij}=1 \mid \mathcal{D}) &= \text{logit } \pi_{ij} \\ &+ \sum_k \log(\bar{L}_{ijk}^1 / L_{ijk}^0), \end{aligned} \quad (4)$$

where $\bar{L}_{ijk}^1 = \int P(\mathcal{D}_{ijk} \mid q, e_{ij}=1) dP(q)$ is the marginal likelihood of patch k ’s outcomes under a true edge and $L_{ijk}^0 = P(\mathcal{D}_{ijk} \mid e_{ij}=0)$. Every update multiplies one grid vector by a likelihood vector, so updates and look-aheads are vectorised dot products.

4.4 Selecting interventions

The value of the current decision for cell (i, j) , with P_{ij} its edge posterior and $\hat{Q}_{ij} = \max_k \mathbb{E}[q_{ijk} \mid \mathcal{D}]$, is

$$V_{ij} = \max(P_{ij} f_i \hat{Q}_{ij} - (1 - P_{ij}) c_{\text{fp}}, 0), \quad (5)$$

the expected contribution of the cell to Eq. 1 under the Bayes decision (accept iff the first term is positive), treating cells as additive. Eq. 1 combines components of one category multiplicatively, so V_{ij} slightly overstates the value of a second cause for an already repaired category; we accept this approximation because it keeps cells independent. For an action $a = (i, j, k, \phi)$ repeated n times, let $V'_{ij}(s)$ be the value after observing s successes. The n -step knowledge gradient is

$$\text{KG}_n(a) = \sum_{s=0}^n P(s \mid \mathcal{D}, a) V'_{ij}(s) - V_{ij}, \quad (6)$$

and CARVE runs the action maximising

$$\text{score}(a) = \max_{n \in \{1,2,4,8\}} \frac{\text{KG}_n(a)}{n c_\phi}. \quad (7)$$

Looking ahead several observations matters because a single binary outcome often cannot flip a decision, in which case $\text{KG}_1 = 0$ even for a valuable cell; maximising over n is the standard remedy (Frazier et al., 2008). Because cells are independent, only the observed cell’s scores change after each outcome, so each step rescores one cell in $O(KG)$ time after an initial pass over all cells.

Decisions and stopping. At any point the method accepts the cells whose current value is positive and applies, for each, the patch with the highest posterior mean quality. It stops early when no action has a score above a small threshold, i.e. when no remaining experiment is expected to be worth its cost.

5 When Do Priors and Cheap Checks Help?

These results analyse an idealised per-cell sequential test, not CARVE’s full policy; they explain which quantities govern the method’s behaviour. We consider a single cell and a single patch with known success probabilities p_1 (true edge) and $p_0 = b$, under a rule that stops when the posterior log-odds leaves $[-A, A]$, $A = \log \frac{1-\delta}{\delta}$. Let $D_1 = \text{KL}(p_1 \parallel p_0)$ and $D_0 = \text{KL}(p_0 \parallel p_1)$ be the expected log-likelihood-ratio drift per observation under each hypothesis. Proofs, which follow Wald’s approximations (Wald, 1945), are in Appendix A.

Proposition 1 (Value of the LLM prior). *Let $|\ell| < A$ and neglect overshoot and stopping at the wrong boundary (whose probability is approximately $\delta e^{-\ell}$ for a true edge and δe^ℓ for a false one). A cell with initial log-odds ℓ then needs $N_1(\ell) = (A - \ell)/D_1$ observations in expectation if it is a true edge and $N_0(\ell) = (A + \ell)/D_0$ if not. Replacing a flat prior ℓ_ρ by LLM-derived log-odds ℓ_{ij} saves, in expectation over cells,*

$$\Delta = \frac{\rho \mathbb{E}[\delta \ell \mid e=1]}{D_1} - \frac{(1 - \rho) \mathbb{E}[\delta \ell \mid e=0]}{D_0}, \quad (8)$$

with $\delta \ell = \ell_{ij} - \ell_\rho$, observations per cell, which is positive exactly when the prior raises true edges more, relative to their base rate and drift, than it raises false ones.

Under Eq. 3, $\ell_{ij} \rightarrow \ell_p$ as $\hat{r} \rightarrow 0$, so $\Delta \rightarrow 0$: a useless analyst costs nothing beyond the flat prior, whereas trusting its labels outright (LLM-only) makes permanent errors. A miscalibrated analyst can make Δ negative, but only by an amount linear in the log-odds shift, after which the evidence takes over. Appendix C compares these predictions with simulated sequential tests.

Proposition 2 (Cheap checks). *Let D_e^S denote the drifts of single-step outcomes, $z \sim \text{Bern}(s p_e + \epsilon(1 - p_e))$. Under the same approximations, a test using only single-step checks reaches the stopping boundary at lower expected cost than one using only full replays if and only if*

$$\frac{c_S}{c_F} < \frac{D_e^S}{D_e}, \quad (9)$$

where e is the true state of the cell.

Since e is unknown, the proposition gives a clear rule only when the cost ratio lies outside the interval spanned by the two critical ratios; inside it, which fidelity is better depends on the posterior, which is what CARVE’s look-ahead weighs.

With our default parameters ($p_1 = 0.54$, $b = 0.05$, $s = 0.85$, $\epsilon = 0.15$) the critical ratio is 0.31 for true edges and 0.44 for false ones; with a weak check ($s = 0.65$, $\epsilon = 0.35$) it falls to 0.05–0.08. Section 6.4 tests both predictions.

6 Controlled Experiments

Real agents do not reveal which component truly caused a failure, so we first evaluate in a testbed where the causal graph is known and the method’s model is deliberately misspecified.

6.1 Setup

Testbed. Each world has $I = 12$ categories with Zipfian frequencies, $J = 20$ components and $K = 3$ patches per cell. Every category has one or two true components and one *decoy* component that wrong attributions prefer, mimicking symptom-level mislabelling. Patch quality is bimodal: with probability 0.3 a patch is bad ($q \sim \text{Beta}(1, 9)$), otherwise $q \sim \text{Beta}(6, 3)$. A trajectory sampled for category i is mislabelled with probability $1 - \lambda = 0.2$, in which case it comes from another category (weighted by frequency) and is repaired only if *its own* edge to j exists, unlike the model’s flat treatment of mislabelled traces. The analyst attributes 20 failures per category with accuracy $r = 0.5$; wrong attributions go to the

decoy with probability 0.6. The analyst’s accuracy is estimated from an audit of 20 attributions. Costs are $c_F = 1$, $c_S = 0.1$, $c_{fp} = 0.02$, and $s = 0.85$, $\epsilon = 0.15$, $b = 0.05$. Section 6.5 relaxes the assumptions the generator shares with the model. Full details are in Appendix B.

Baselines. *LLM-only* accepts, for every category, the most-attributed component and any component holding at least 40% of the attributions, with the first patch, i.e. it acts on an induced taxonomy without verification. *Replay-each* tests attributed hypotheses in order of attribution count with three full replays of the first patch and accepts a hypothesis if at least two succeed, mirroring do-then-verify methods such as DoVer; *Replay-each, 3 patches* gives each hypothesis two replays of each of three patches and accepts if any patch succeeds twice; *Sequential-each* replays each hypothesis, cycling through patches, until CARVE’s own posterior leaves $[0.05, 0.95]$. Untested hypotheses fall back to LLM-only, except for *Sequential-each*, which uses the Bayes decision. *Thompson sampling* samples (e, q) from the posterior and replays the cell and patch with the largest sampled repair. *Uncertainty sampling* replays the cell with the largest frequency-weighted uncertainty $f_i \min(P_{ij}, 1 - P_{ij})$, cycling through patches, at full fidelity; *Uncertainty + cheap checks* first runs three single-step checks per (cell, patch) and then confirms with full replays. All baselines except LLM-only and the *Replay-each* variants share CARVE’s posterior and decision rule, so comparisons among them isolate the selection rule.

Metrics. Net gain (Eq. 1) as a fraction of the oracle, and edge F1. We report means and standard errors over 50 worlds for the main experiment and 30 for the others; all methods see the same worlds, so differences are paired.

6.2 Main results

Figure 2 and Table 1 show that CARVE is best at every budget. With 50 units it recovers 79% of the oracle gain; the strongest baseline, uncertainty sampling with the same cheap checks, recovers 66% and needs about twice the budget (100 units, 78%) to match it. The gap narrows at large budgets (90% vs. 88% at 200), as every reasonable method eventually resolves the frequent categories.

Where the gain comes from. Two comparisons separate the selection rule from the cheap checks.

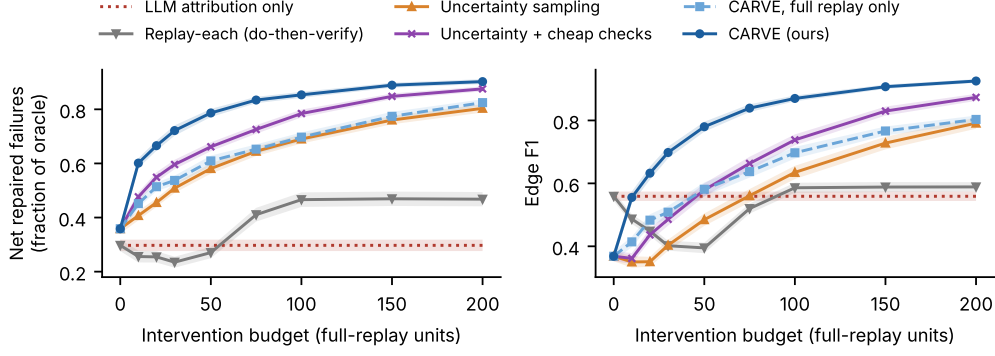


Figure 2: Net repaired failure mass (left) and edge F1 (right) against intervention budget, 50 worlds, shaded ± 1 s.e.

Method	Budget (full-replay units)			
	20	50	100	200
LLM attribution only	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021
Replay-each (do-then-verify)	0.255 $\pm$ 0.021	0.270 $\pm$ 0.021	0.466 $\pm$ 0.027	0.468 $\pm$ 0.026
Replay-each, 3 patches	0.301 $\pm$ 0.025	0.317 $\pm$ 0.026	0.401 $\pm$ 0.026	0.612 $\pm$ 0.026
Sequential-each (adaptive stopping)	0.426 $\pm$ 0.016	0.522 $\pm$ 0.018	0.643 $\pm$ 0.016	0.813 $\pm$ 0.014
Thompson sampling	0.407 $\pm$ 0.020	0.419 $\pm$ 0.020	0.500 $\pm$ 0.017	0.625 $\pm$ 0.014
Uncertainty sampling	0.455 $\pm$ 0.016	0.581 $\pm$ 0.019	0.690 $\pm$ 0.017	0.804 $\pm$ 0.016
Uncertainty + cheap checks	0.549 $\pm$ 0.018	0.662 $\pm$ 0.016	0.784 $\pm$ 0.015	0.876 $\pm$ 0.009
CARVE (all components)	0.666 $\pm$ 0.019	0.787 $\pm$ 0.015	0.853 $\pm$ 0.011	0.902 $\pm$ 0.009
full replay only	0.514 $\pm$ 0.016	0.609 $\pm$ 0.020	0.697 $\pm$ 0.017	0.825 $\pm$ 0.012
myopic ($n=1$ look-ahead)	0.591 $\pm$ 0.016	0.652 $\pm$ 0.014	0.706 $\pm$ 0.016	0.729 $\pm$ 0.014
uninformed prior	0.347 $\pm$ 0.019	0.474 $\pm$ 0.017	0.631 $\pm$ 0.015	0.800 $\pm$ 0.012
one patch per cell	0.571 $\pm$ 0.019	0.636 $\pm$ 0.020	0.673 $\pm$ 0.019	0.688 $\pm$ 0.020
no label-noise term	0.663 $\pm$ 0.020	0.772 $\pm$ 0.018	0.864 $\pm$ 0.012	0.922 $\pm$ 0.006

Table 1: Net gain as a fraction of the oracle (mean $\pm$ s.e., 50 worlds). The lower block ablates CARVE.

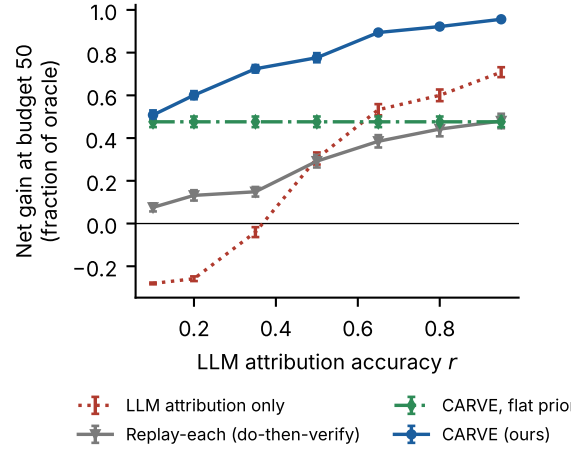


Figure 3: Net gain at budget 50 as the analyst’s attribution accuracy r varies (30 worlds, ± 1 s.e.).

With full replays only, CARVE beats uncertainty sampling by 1–6 points (e.g. 0.609 vs. 0.581 at budget 50): the knowledge-gradient rule alone helps only modestly. Given cheap checks, its advantage grows to 7–13 points (0.787 vs. 0.662 at 50), because it decides per action whether a check is informative enough for its price and stops checking a cell once checks can no longer change the decision, whereas a fixed check-then-confirm schedule wastes checks on resolved cells. Most of CARVE’s advantage is therefore in *how it uses* cheap checks, which also makes their calibration critical (§6.5).

Verification baselines. LLM-only is stuck at 30% because it accepts decoys and bad first patches. Replay-each is initially *worse* than not verifying: with $p_1 \approx 0.54$, requiring two successes in three replays rejects about 44% of true edges even when the patch is good, and its fixed per-hypothesis cost exhausts small budgets on frequent but already-certain hypotheses. Testing three patches (61% at 200) or stopping adaptively (81% at 200) helps, but both remain well below CARVE at small budgets (32% and 52% at 50). Thompson sampling spends most replays on cells it already believes in and reaches only 63% at 200.

Ablations. Removing single-step checks costs 15–18 points at small budgets. Restricting the look-ahead to one observation is below the full method at every budget and plateaus at 73%, because many cells cannot be flipped by a single outcome and are never revisited. A flat prior costs 32 points at budget 20 and still 10 points at 200. With one patch per cell the method plateaus at 69%: it cannot tell a wrong component from a bad patch and rejects true edges for good. Dropping the label-noise term makes no difference at the default label accuracy $\lambda = 0.8$ (it is even slightly better at large budgets, 0.922 vs. 0.902 at 200), but it matters when labels are worse (§6.5).

6.3 When does the LLM prior help?

Figure 3 varies the analyst’s accuracy r . As Proposition 1 suggests, CARVE improves monotonically with r and is never below the flat-prior variant: at $r = 0.1$ it scores 0.51 ± 0.02 against 0.48 ± 0.03 .

Condition (budget 50)	Unc. + checks	CARVE
Default testbed	0.656 $\pm$ 0.022	0.776 $\pm$ 0.021
Label accuracy $\lambda = 0.5$	0.513 $\pm$ 0.023	0.688 $\pm$ 0.023
Analyst overconfident ($\hat{r}+0.3$)	0.752 $\pm$ 0.018	0.857 $\pm$ 0.014
Analyst underconfident ($\hat{r}-0.3$)	0.495 $\pm$ 0.023	0.653 $\pm$ 0.032
Judge fooled by bad patches	0.579 $\pm$ 0.022	0.377 $\pm$ 0.027
Regression cost $c_{fp} = 0.005$	0.711 $\pm$ 0.022	0.798 $\pm$ 0.016
Regression cost $c_{fp} = 0.05$	0.643 $\pm$ 0.024	0.793 $\pm$ 0.019
Regression cost $c_{fp} = 0.1$	0.624 $\pm$ 0.023	0.823 $\pm$ 0.018

Table 2: Violations of the model’s assumptions (30 worlds). The judge-fooled row is the one setting where CARVE loses.

The prior is not exactly flat there (the decoy still receives about twice the base rate), but its shifts are small, so evidence quickly dominates. Acting on attributions directly is harmful below $r \approx 0.35$, where LLM-only has a *negative* net gain (regressions outweigh repairs), and even at $r = 0.95$ it reaches only 71% of the oracle against 96% for CARVE, since it cannot detect bad patches. Replay-each improves with r and only matches the flat-prior variant of CARVE at $r = 0.95$ (0.48 vs. 0.48).

6.4 When do cheap checks help?

Figure 5 (Appendix D) is qualitatively consistent with Proposition 2. With the default check, the benefit of cheap checks shrinks as their cost grows and vanishes between 0.35 and 0.5 (0.64 ± 0.02 vs. 0.61 ± 0.03 at 0.35, overlapping; 0.59 vs. 0.61 at 0.5), in line with the predicted critical ratios of 0.31 and 0.44. At cost 0.1, a check of quality $s - \epsilon = 0.5$ (critical ratio 0.14–0.22) still helps, while one of quality 0.3 (critical ratio 0.05–0.08) brings no gain. At cost 0.5 CARVE is slightly below its full-replay-only variant although its action set contains every full-replay action, which shows that the greedy rule is not exactly cost-optimal when the two fidelities are nearly equally efficient.

6.5 Robustness

The generator shares several assumptions with the model: check errors depend only on the full outcome, cells are independent, the analyst’s accuracy is estimated without bias, and the regression cost is small. Table 2 and Appendix D relax each in turn.

Worse labels. With label accuracy $\lambda = 0.5$ both methods lose ground, and the label-noise term now matters: CARVE scores 0.688, against 0.588 without the term and 0.513 for the baseline.

Biased analyst audit. If the audit overstates the analyst’s accuracy by 0.3, CARVE actually im-

proves (0.857), because a stronger prior concentrates the budget and the evidence corrects the decoys. Understating it by 0.3 costs 12 points (0.653), still above the baseline. Under-trusting a decent analyst is thus the more costly error in this testbed.

Regression cost. Raising c_{fp} from 0.005 to 0.1 leaves CARVE between 0.78 and 0.82, while LLM-only falls from 0.39 to -0.13 : the more costly a wrong change, the more verification is worth.

Joint causes. When 30% of categories need two components changed together (a single patch then repairs only a fifth as often), violating cell independence, CARVE drops from 0.787 to 0.661 at budget 50 but stays ahead of uncertainty sampling with checks (0.558; Table 5).

Correlated check errors. The one setting where CARVE loses is when the single-step judge is fooled by plausible-looking bad patches (false-positive rate 0.6 for patches with $q < 0.3$). CARVE then trusts checks that systematically pass bad patches and falls to 0.377, below every method that relies on full replays (0.58–0.61). Modelling the dependence (a quality-dependent false-positive rate in the likelihood) restores it to 0.584, on par with full replays, at a cost of 9 points when the judge is not fooled (0.684 vs. 0.776). Whether real judges behave this way is therefore the most important quantity to measure before relying on cheap checks.

Nuisance parameters and scale. Misspecifying b , λ or the patch-quality prior changes the result by at most 5 points (Table 6, Appendix D); misstating the check’s sensitivity and false-positive rate costs 6–14 points. On a larger graph ($I = 20$, $J = 50$; Table 4) the gap widens: at budget 50 CARVE reaches 78% of the oracle against 60% for uncertainty sampling with checks.

6.6 Inside a self-improvement loop

Finally we embed each method in a loop. The agent starts at 40% success. In each round the analyst attributes 240 new failures, sampled in proportion to the current failure distribution; the method spends 40 units; accepted patches are applied, which repairs failures (true edges) or adds a regression of c_{fp} times the current failure mass (false edges); and the failure distribution is recomputed, so priorities shift towards categories that remain. Posteriors carry over between rounds.

Figure 4 shows that CARVE reaches 80% success after one round and 85% after two, close to its

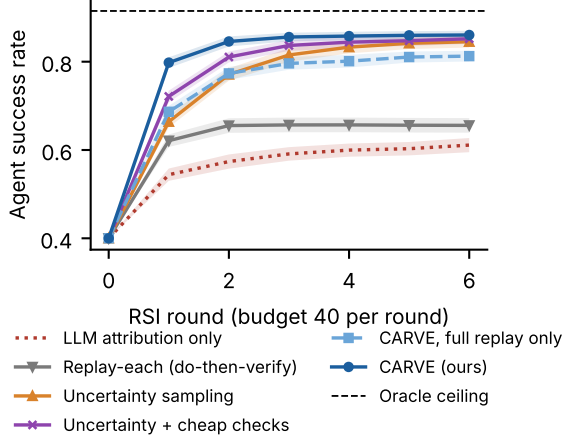


Figure 4: Agent success over six rounds of attribute-verify-patch, with a budget of 40 units per round (30 worlds, ± 1 s.e.).

plateau of 86% (the oracle ceiling is 91.5%). Uncertainty sampling with checks needs four rounds to reach 84% and ends at 85%; without checks it needs six. LLM-only and Replay-each plateau at 61% and 66%: once their accepted false edges and rejected true edges are fixed in place, new attributions do not correct them.

7 Real-Agent Study

Environment. ShopDesk is a customer-service environment with a seeded database, six deterministic tools and exactly checkable answers (refund amounts, order counts, spend totals), so replay from any step is exact. The agent is a ReAct-style tool-use loop (Yao et al., 2023) assembled from 11 named components (four system-prompt sections, six tool descriptions and the step limit), and every step records which components shaped it.

Ground truth by fault injection. Following the fault-injection idea of Zhang et al. (2026), each training task runs under exactly one of five injected faults or none: a tool description that misstates price units, a list tool whose description omits pagination, a format instruction that rounds to whole dollars, a planning instruction that hard-codes the wrong refund policy, and a step limit that is too small. The responsible component of every failure is therefore known. After the analyst attributes failures and induces a taxonomy in TnT-LLM style (Wan et al., 2024), an edge is true when at least 30% of a category’s failures come from that com-

ponent’s fault.

Protocol. All methods share the same attributions, taxonomy and token budget. Single-step checks use an LLM judge (Zheng et al., 2023); the analyst generates three patches per cell on demand, charged to the method; cost units come from measured tokens. We report edge precision, recall and held-out success after applying accepted patches.

Status. [TODO: The real-agent experiments have not yet been run: the environment used to prepare this draft had no model API access. The pipeline is implemented and tested end to end with a mock model (agent_exp/); results, the measured cost ratio c_S/c_F , the analyst’s measured attribution accuracy and a qualitative analysis of accepted patches will be added here. No numbers in this section should be read as results.]

8 Discussion

What the analyst LLM is for. Our results suggest moving LLM attribution from judge to prior: good attributions focus the budget, bad ones fall back to an uninformed prior, whereas acting on them directly causes regressions that later rounds do not undo. Modelling patch quality separately, with several patches per cell, is what keeps a bad first patch from rejecting a true edge for good.

Cheap checks are a double-edged sword. CARVE’s advantage over the strongest baseline comes mostly from spending cheap checks well. The same reliance makes it fragile when a judge systematically approves plausible but wrong patches, a pattern analogous to patch overfitting in program repair (Smith et al., 2015). In practice, the check’s false-positive rate should be measured separately for patches that later fail full replay, and a final full-replay confirmation before applying a patch is a cheap safeguard.

9 Conclusion

We framed verification in failure-driven self-improvement as budgeted learning of a causal category-component graph. CARVE treats LLM attributions as a prior, separates causes from patch quality, and allocates replays by expected value per unit cost. In controlled experiments it matches the strongest baseline with about half the budget, but it relies on cheap checks whose errors must not correlate with patch quality; measuring that correlation on real agents is the main open item.

Limitations

The quantitative results in this draft come from a simulated testbed. Although the generator is deliberately richer than the model (systematic decoy attributions, bimodal patch quality, mislabelled traces that can still be repaired), real agents may violate the model in ways we did not simulate. Our stress tests cover joint causes, correlated check errors, biased audits and worse labels, but not patches that help one category while hurting another beyond a constant regression cost, nor non-stationary edges as the agent changes. The prior of Eq. 3 is a heuristic, and its parameter d is set to the generator’s mean. The method also assumes that failures can be replayed from the attributed step; this holds for deterministic tools and sandboxes but not for live web environments. The single-step check relies on an LLM judge whose quality must be calibrated; Table 2 shows that if its errors correlate with patch quality, CARVE can do worse than methods that use full replays only. Finally, the propositions use Wald’s approximations and ignore overshoot and errors at the boundary; they are guides to when the method helps, not guarantees.

Ethics Statement

Automated self-improvement changes an agent’s behaviour without a human reviewing each change. CARVE makes these changes more reliable by requiring interventional evidence before acceptance, but accepted patches should still be reviewed before deployment in settings where errors carry real costs. Our environment uses synthetic data only.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*.

Jean-Yves Audibert, Sébastien Bubeck, and Rémi Munos. 2010. Best arm identification in multi-armed bandits. In *Proceedings of the 23rd Conference on Learning Theory*.

Akash Bonagiri, Devang Borkar, Gerard Janno Andrias, Setareh Rafatirad, and Houman Homayoun.

2026. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*.

Mert Cemri, Andrei Cojocaru, Melissa Pan, Shu Liu, Shubham Agarwal, Alexander Krentsel, Jay Tang, Kannan Ramchandran, Joseph E. Gonzalez, Matei Zaharia, Alexandros G. Dimakis, and Ion Stoica. 2026. Fantastic adaptive taxonomies and how to use them. *arXiv preprint arXiv:2607.16387*.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.

Kathryn Chaloner and Isabella Verdinelli. 1995. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304.

Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*.

Peter I. Frazier and Warren B. Powell. 2010. Paradoxes in learning and the marginal value of information. *Decision Analysis*, 7(4):378–403.

Peter I. Frazier, Warren B. Powell, and Savas Dayanik. 2008. A knowledge-gradient policy for sequential information collection. *SIAM Journal on Control and Optimization*, 47(5):2410–2439.

Alain Hauser and Peter Bühlmann. 2014. Two optimal strategies for active learning of causal models from interventional data. *International Journal of Approximate Reasoning*, 55(4):926–939.

Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *International Conference on Learning Representations*.

Kirthevasan Kandasamy, Gautam Dasarathy, Jeff Schneider, and Barnabás Póczos. 2016. The multi-fidelity multi-armed bandit. In *Advances in Neural Information Processing Systems*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*.

Finnian Lattimore, Tor Lattimore, and Mark D. Reid. 2016. Causal bandits: Learning good interventions via causal inference. In *Advances in Neural Information Processing Systems*.

757	Jiazheng Li, Emine Yilmaz, Bei Chen, and Dieu-	Edward K. Smith, Earl T. Barr, Claire Le Goues, and	810
758	Thu Le. 2026. Towards self-improving error di-	Yuriy Brun. 2015. Is the cure worse than the disease?	811
759	agnosis in multi-agent systems. <i>arXiv preprint</i>	overfitting in automated program repair. In <i>Proceed-</i>	812
760	<i>arXiv:2604.17658</i> .	<i>ings of the 10th Joint Meeting on Foundations of</i>	813
		<i>Software Engineering</i> .	814
761	Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok,	Simon Tong and Daphne Koller. 2001. Active learning	815
762	Daniel Guo, Sahil Arun Nale, Charles Fleming,	for structure in Bayesian networks. In <i>Proceedings of</i>	816
763	and Guang Cheng. 2026. REFLECT: Intervention-	<i>the 17th International Joint Conference on Artificial</i>	817
764	supported error attribution for silent failures in LLM	<i>Intelligence</i> .	818
765	agent traces. <i>arXiv preprint arXiv:2606.09071</i> .		
766	Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tian-	Rakshith Vasudev, Melisa Russak, Dan Bikel, and	819
767	wei Yue, Chi Wang, Jian Kang, Qingyun Wu, and	Waseem Alshikh. 2026. Accurate failure prediction	820
768	Huazheng Wang. 2026. Who&when pro: Can LLMs	in agents does not imply effective failure prevention.	821
769	really attribute failures in AI agents? <i>arXiv preprint</i>	<i>arXiv preprint arXiv:2602.03338</i> .	822
770	<i>arXiv:2607.09996</i> .		
771	Andrea Locatelli, Maurilio Gutzeit, and Alexandra Car-	Abraham Wald. 1945. Sequential tests of statistical	823
772	pentier. 2016. An optimal algorithm for the thresh-	hypotheses. <i>The Annals of Mathematical Statistics</i> ,	824
773	olding bandit problem. In <i>Proceedings of the 33rd</i>	16(2):117–186.	825
774	<i>International Conference on Machine Learning</i> .		
775	Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qing-	Mengting Wan, Tara Safavi, Sujay Kumar Jauhar, Yujin	826
776	wei Lin, Saravan Rajmohan, and Dongmei Zhang.	Kim, Scott Counts, Jennifer Neville, Siddharth Suri,	827
777	2025. DoVer: Intervention-driven auto debug-	Chirag Shah, Ryen W. White, Longqi Yang, Reid	828
778	ging for LLM multi-agent systems. <i>arXiv preprint</i>	Andersen, Georg Buscher, Dhruv Joshi, and Nagu	829
779	<i>arXiv:2512.06749</i> .	Rangan. 2024. TnT-LLM: Text mining at scale with	830
		large language models. In <i>Proceedings of the 30th</i>	831
		<i>ACM SIGKDD Conference on Knowledge Discovery</i>	832
		<i>and Data Mining</i> .	833
780	Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler	Xiaojun Wu, Cehao Yang, Honghao Liu, Xueyuan Lin,	834
781	Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon,	Wenjie Zhang, Zhichao Shi, Xuhui Jiang, Chengjin	835
782	Nouha Dziri, Shrimai Prabhumoye, Yiming Yang,	Xu, Jia Li, and Jian Guo. 2026. Bayesian-Agent:	836
783	Shashank Gupta, Bodhisattwa Prasad Majumder,	Posterior-guided skill evolution across LLM agent	837
784	Katherine Hermann, Sean Welleck, Amir Yazdan-	harnesses. <i>arXiv preprint arXiv:2606.08348</i> .	838
785	bakhsh, and Peter Clark. 2023. Self-refine: Iterative		
786	refinement with self-feedback. In <i>Advances in Neu-</i>	Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu,	839
787	<i>ral Information Processing Systems</i> .	Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024.	840
788	Judea Pearl. 2009. <i>Causality: Models, Reasoning, and</i>	Large language models as optimizers. In <i>Internat-</i>	841
789	<i>Inference</i> , 2nd edition. Cambridge University Press.	<i>ional Conference on Learning Representations</i> .	842
790	Matthias Poloczek, Jialei Wang, and Peter I. Frazier.	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak	843
791	2017. Multi-information source optimization. In	Shafraan, Karthik Narasimhan, and Yuan Cao. 2023.	844
792	<i>Advances in Neural Information Processing Systems</i> .	ReAct: Synergizing reasoning and acting in language	845
793	Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chen-	models. In <i>International Conference on Learning</i>	846
794	guang Zhu, and Michael Zeng. 2023. Automatic	<i>Representations</i> .	847
795	prompt optimization with “gradient descent” and	Mert Yuksekgonul, Federico Bianchi, Joseph Boen,	848
796	beam search. In <i>Proceedings of the 2023 Confer-</i>	Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and	849
797	<i>ence on Empirical Methods in Natural Language</i>	James Zou. 2025. Optimizing generative AI by back-	850
798	<i>Processing</i> .	propagating language model feedback. <i>Nature</i> , 639.	851
799	Daniel Russo. 2016. Simple Bayesian algorithms for	Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying	852
800	best arm identification. In <i>Proceedings of the 29th</i>	and isolating failure-inducing input. <i>IEEE Transac-</i>	853
801	<i>Conference on Learning Theory</i> .	<i>tions on Software Engineering</i> , 28(2):183–200.	854
802	Jaineet Shah. 2026. Causal agent replay: Counterfactual	Guibin Zhang, Junhao Wang, Junjie Chen, Wangchun-	855
803	attribution for LLM-agent failures. <i>arXiv preprint</i>	shu Zhou, Kun Wang, and Shuicheng Yan. 2026.	856
804	<i>arXiv:2606.08275</i> .	AgenTracer: Who is inducing failure in the LLM	857
		agentic systems? In <i>International Conference on</i>	858
		<i>Learning Representations</i> . ArXiv:2509.03312.	859
805	Noah Shinn, Federico Cassano, Ashwin Gopinath,	Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange,	860
806	Karthik Narasimhan, and Shunyu Yao. 2023. Re-	and Jeff Clune. 2025a. Darwin Gödel machine:	861
807	flexion: Language agents with verbal reinforcement	Open-ended evolution of self-improving agents.	862
808	learning. In <i>Advances in Neural Information Pro-</i>	<i>arXiv preprint arXiv:2505.22954</i> .	863
809	<i>cessing Systems</i> .		

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025b. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*. ArXiv:2505.00212.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. 2026. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*.

A Proofs

Proposition 1. Let $L_t = \ell + \sum_{u \leq t} \log \frac{P(o_u|e=1)}{P(o_u|e=0)}$ be the posterior log-odds after t observations. Under $e = 1$, the increments are i.i.d. with mean $D_1 = \text{KL}(p_1||p_0) > 0$. The rule stops at $N = \inf\{t : L_t \notin (-A, A)\}$. By Wald’s identity, $\mathbb{E}_1[L_N - \ell] = D_1 \mathbb{E}_1[N]$. Wald’s approximations give $P_1(L_N \leq -A) \approx e^{-(A+\ell)} \approx \delta e^{-\ell}$ and $P_0(L_N \geq A) \approx \delta e^{\ell}$. Neglecting overshoot and this probability (which requires $\delta e^{|\ell|} \ll 1$), $L_N = A$, hence $\mathbb{E}_1[N] = (A - \ell)/D_1$; if $\ell \geq A$ no observation is taken, which is why we assume $|\ell| < A$. Symmetrically, under $e = 0$ the drift is $-D_0$ and $\mathbb{E}_0[N] = (A + \ell)/D_0$. Taking the difference between ℓ_ρ and ℓ_{ij} and averaging over cells with $P(e=1) = \rho$ gives Δ . When the patch quality q is unknown, outcomes are exchangeable rather than i.i.d. under the marginal likelihood, and Wald’s identity does not apply directly; conditioning on q recovers the result with $D_1 = D_1(q)$, which approaches zero for a bad patch. This is the

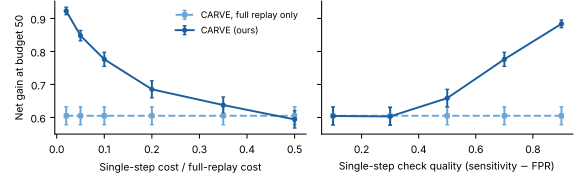


Figure 5: Effect of the single-step check’s cost (left, quality fixed at $s=0.85$, $\epsilon=0.15$) and quality (right, cost fixed at 0.1) on net gain at budget 50.

formal reason a single bad patch can stall verification and why CARVE keeps several patches per cell.

Proposition 2. By the same argument a single-step test needs $(A \mp \ell)/D_e^S$ observations at cost c_S each, and a full-replay test needs $(A \mp \ell)/D_e$ at cost c_F . The single-step test is cheaper iff $c_S/D_e^S < c_F/D_e$. Since the single-step outcome is a garbling of the full outcome, the data-processing inequality gives $D_e^S \leq D_e$, so the critical ratio is at most one.

B Simulation Details

Category frequencies are $f_i \propto 1/i$. For each category the number of true components is uniform on $\{1, 2\}$ and the decoy is a further distinct component. The audit estimate $\hat{r}$ is a binomial proportion from 20 audited attributions. The prior uses $d = 1.5$. The model’s grid has 101 points; look-ahead horizons are $\{1, 2, 4, 8\}$; the stopping threshold is 10^{-6} . Replay-each uses three replays and accepts with at least two successes. Uncertainty sampling switches to an untried patch once the current patch has three full observations. In the loop experiment, 240 attributions per round are allocated to categories by a multinomial draw with the current failure distribution. All randomness is seeded; the code reproduces every number in this paper with python -m sim.experiments all.

C Checking Proposition 1

Table 3 compares the predicted expected number of observations with simulated sequential tests ($p_1 = 0.543$, $p_0 = 0.05$, $\delta = 0.02$, 4000 runs per row). The predictions capture the direction and approximate size of the effect of the initial log-odds; they are least accurate when $|\ell|$ is large, where overshoot and wrong-boundary exits (last column) are no longer negligible.

D Additional Results

ℓ	e	predicted N	simulated N	wrong exit
-3	1	7.17	5.94	0.242
-3	0	1.55	2.54	0.000
-2	1	6.13	6.28	0.117
-2	0	3.28	3.73	0.001
-1	1	5.09	5.79	0.047
-1	0	5.02	5.21	0.003
+0	1	4.05	4.76	0.012
+0	0	6.75	7.28	0.009
+1	1	3.01	4.27	0.006
+1	0	8.48	8.58	0.015
+2	1	1.97	2.97	0.001
+2	0	10.22	10.23	0.068

Table 3: Proposition 1: predicted vs. simulated observations to a decision.

Method	50	100	300
LLM attribution only	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033
Replay-each (do-then-verify)	0.224 $\pm$ 0.034	0.400 $\pm$ 0.037	0.531 $\pm$ 0.032
Uncertainty sampling	0.502 $\pm$ 0.022	0.637 $\pm$ 0.020	0.822 $\pm$ 0.015
Uncertainty + cheap checks	0.603 $\pm$ 0.019	0.746 $\pm$ 0.018	0.873 $\pm$ 0.013
CARVE, full replay only	0.613 $\pm$ 0.013	0.710 $\pm$ 0.012	0.834 $\pm$ 0.010
CARVE (ours)	0.780 $\pm$ 0.016	0.860 $\pm$ 0.011	0.933 $\pm$ 0.008

Table 4: Net gain on a larger graph ($I=20$, $J=50$), 20 worlds.

Method	20	50	100	200
LLM attribution only	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028
Replay-each	0.202 $\pm$ 0.027	0.207 $\pm$ 0.023	0.348 $\pm$ 0.033	0.362 $\pm$ 0.036
Thompson sampling	0.325 $\pm$ 0.031	0.343 $\pm$ 0.032	0.398 $\pm$ 0.032	0.484 $\pm$ 0.028
Uncertainty sampling	0.345 $\pm$ 0.036	0.465 $\pm$ 0.035	0.544 $\pm$ 0.035	0.662 $\pm$ 0.029
Uncertainty + cheap checks	0.423 $\pm$ 0.037	0.558 $\pm$ 0.035	0.615 $\pm$ 0.034	0.685 $\pm$ 0.030
CARVE (ours)	0.573 $\pm$ 0.029	0.661 $\pm$ 0.029	0.701 $\pm$ 0.027	0.745 $\pm$ 0.026

Table 5: Net gain when 30% of categories need two components changed together (30 worlds).

Model assumption (true value)	Net gain
All nuisance parameters as in the generator	0.776 $\pm$ 0.021
Spurious rate b : 0.01 (0.05)	0.811 $\pm$ 0.012
Spurious rate b : 0.15 (0.05)	0.826 $\pm$ 0.016
Label accuracy λ : 1.0 (0.8)	0.768 $\pm$ 0.022
Label accuracy λ : 0.6 (0.8)	0.794 $\pm$ 0.017
Check s/ϵ : .95/.05 (.85/.15)	0.717 $\pm$ 0.016
Check s/ϵ : .70/.30 (.85/.15)	0.636 $\pm$ 0.021
Patch prior Beta(1, 1) (Beta(2, 2))	0.811 $\pm$ 0.015
Patch prior Beta(5, 2) (Beta(2, 2))	0.777 $\pm$ 0.020

Table 6: CARVE with misspecified nuisance parameters, budget 50, 30 worlds. Several “wrong” values do slightly better than the generator’s, a reminder that the model is misspecified in other ways too.